

INTERNATIONAL BLACK SEA UNIVERSITY
School of Computer Science · Computer Science Program

A.L.F.R.E.D.

Adaptive Local-First Routing and Execution Distillation
for Small Language Models

David Zoidze

Bachelor's Thesis in Computer Science

Supervisor: **Giorgi Merabishvili**

Tbilisi · July 2026

The problem

How much language model does reliable on-device *action* actually require?

WHY ON-DEVICE

Privacy — action requests carry personal context that should never leave the device

Cost & latency — a local model answers in milliseconds, with no round-trip and no per-call bill

Availability — the device should still work offline

Google (Gemini Nano / AICore) and Apple (Apple Intelligence) already make this bet.

THE SQUEEZE

Small model, free-form — unreliable executor: fails to act, or invents commands from pretraining priors (30% binding)

Large model — does not fit on the phone, an order of magnitude slower, costs per call — and is still wrong on device conventions

The interesting design space: a tiny model that does not have to be clever, paired with something that supplies the knowledge it lacks.

Actions have a single correct outcome — either the alarm is set for 7:00 or it is not.

Thesis statement & research questions

On the repeatable, deterministic head of on-device tasks, **distillation substitutes for scale**.

A 2B executor handed a distilled symbolic pattern matches a 17× larger free-form agent in binding accuracy, at one on-device forward pass. The remaining open limit is routing, not execution.

FIVE RESEARCH QUESTIONS

RQ1 Scale vs. distillation — how much model does reliable binding need — can a pattern close the gap?

RQ2 The spectrum — does the reflexer's value change with API difficulty — efficiency win or accuracy win?

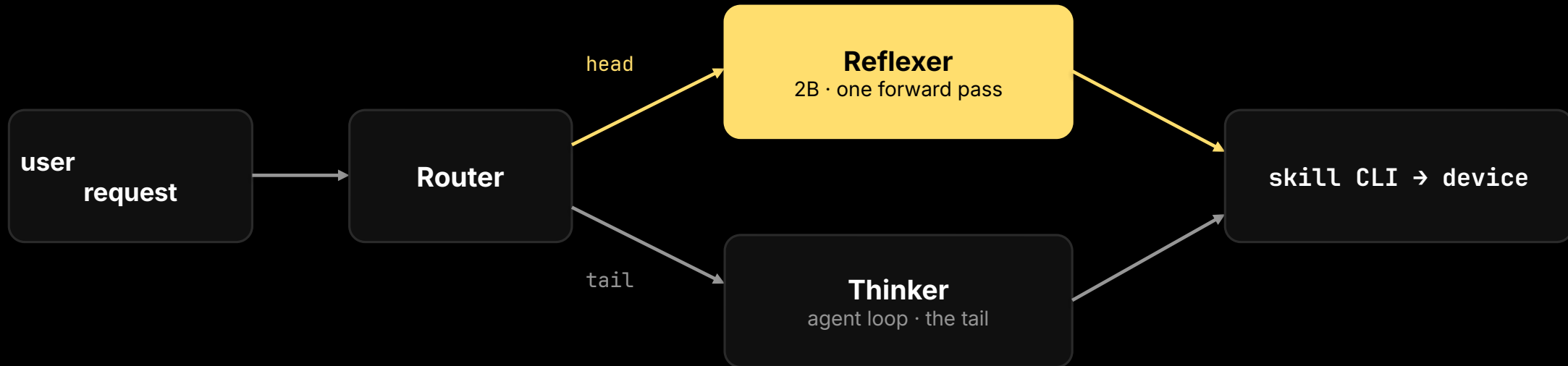
RQ3 The limit of scale — are there difficulties model scale cannot solve but a distilled pattern can?

RQ4 Teacher quality — how much does the choice of teacher (cloud vs. local) propagate to the student?

RQ5 Coverage & routing — what fraction of real use-cases does this address — where is the bottleneck?

System design: two tiers, one router

Most on-device requests do not need a model to *reason*; they need a model to *fill in a blank*.



- ▶ **Reflexer** — an executioner, not a generator — extracts slots, fills a fixed command template, emits one shell command; escalates on any failure
- ▶ **Thinker** — a general ReAct agent for the tail — multi-intent, planning, undistilled intents; built on the Pi coding-agent framework
- ▶ **Skills** — deep modules behind a CLI — small intent-shaped vocabulary (clock timer -duration 10m) hiding messy device mechanism

Symbolic distillation: teacher → pattern → student

OFFLINE — once per intent

ON-DEVICE — every request



- ▶ **Distill into a symbolic artifact, not student weights** — inspectable, plug-and-play (adding a capability = adding a JSON file), auditable, composable
- ▶ **Two regimes** — cold-start (teacher sees only docs + schema) vs. session-grounded (teacher reads the real device)
- ▶ **Integrity by construction** — the teacher never sees evaluation tasks or gold outcomes; a minted pattern is frozen, never edited after failures

Same teacher→student axis as classical distillation (Hinton et al.), different target: **knowledge into an artifact the user can read.**

Methodology: measuring only what is claimed

BINDING-ONLY

Isolate execution from routing

routing assumed perfect: reflexer gets the gold pattern, thinker gets the right docs — only binding is under test; routing is measured separately

END-STATE ORACLE

Success = final device state

requested keys reach target values and nothing else changed (no collateral edits); catches “right-looking command, wrong outcome”

BENCHMARKS

Three difficulty brackets

409 expansion atoms across 7 skills · SET-1 clean settings API (30 tasks) · SET-2 realistic Android API, 32 settings, non-obvious encodings (48 tasks)

INTEGRITY PROTOCOL

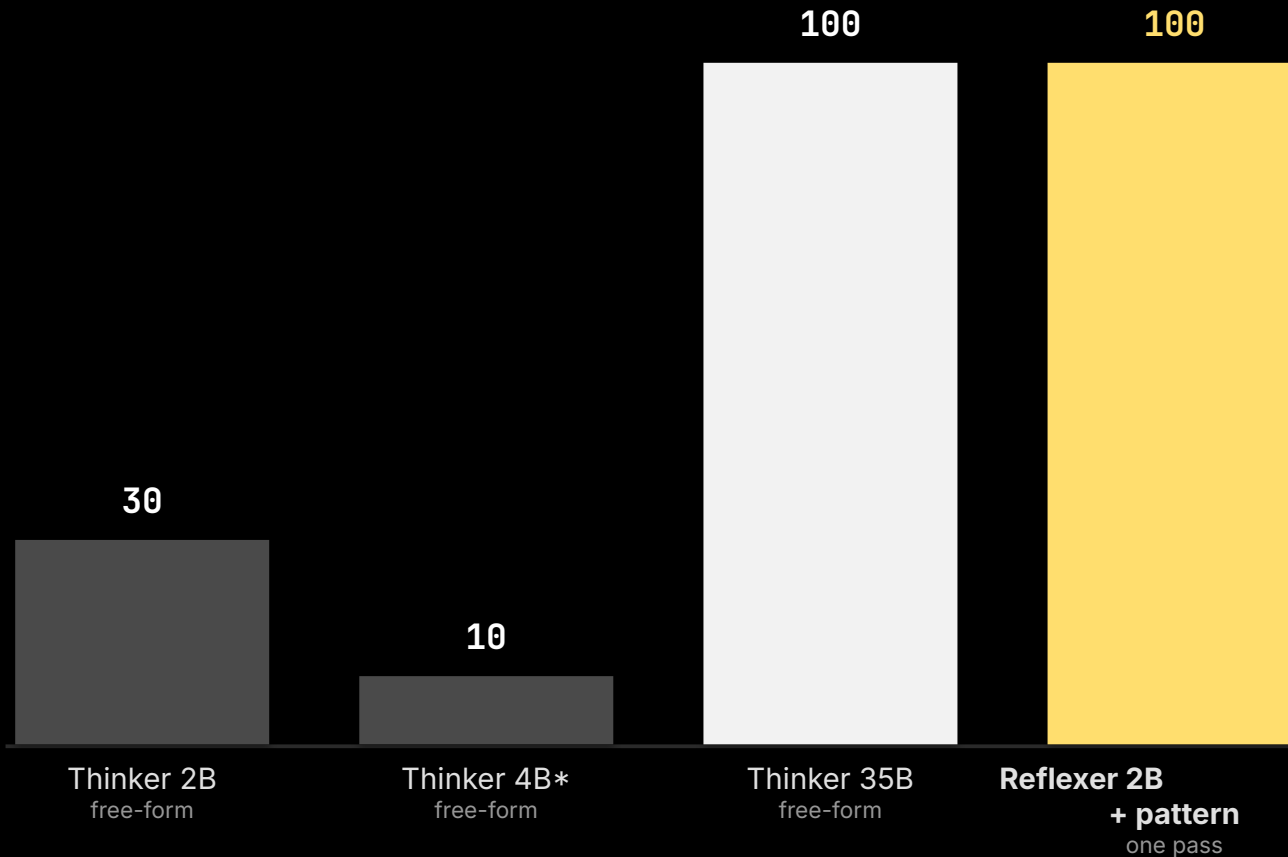
Rules stated before results

no fabricated numbers (every figure cites a saved run) · no benchmark leakage into patterns · sandboxed, destroyed-after-use state · sequential runs

All models Q4-quantized GGUF (phone-class memory) · served by llama.cpp · agent loop driven by Pi

Result 1 — Distillation substitutes for scale

Binding accuracy, balanced-10, routing held fixed (binding-only).



*4B at ctx 4000 (confound) — scale alone is non-monotonic

100% = 100%

the 2B reflexer with a distilled pattern equals the 35B free-form thinker — at ~17× fewer parameters

<1 s vs 6.4 s

one on-device forward pass (~0.3k tokens) vs an off-device agent loop (~1.3k tokens)

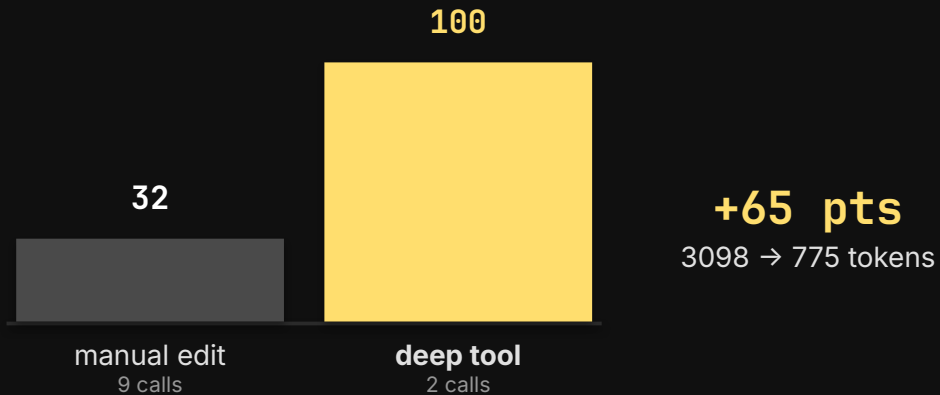
72/72 vs 14/72

the effect persists on the full atom set: reflexer 100%, unaided 2B thinker 19.4%

Result 2 — Scale buys discovery, not convention

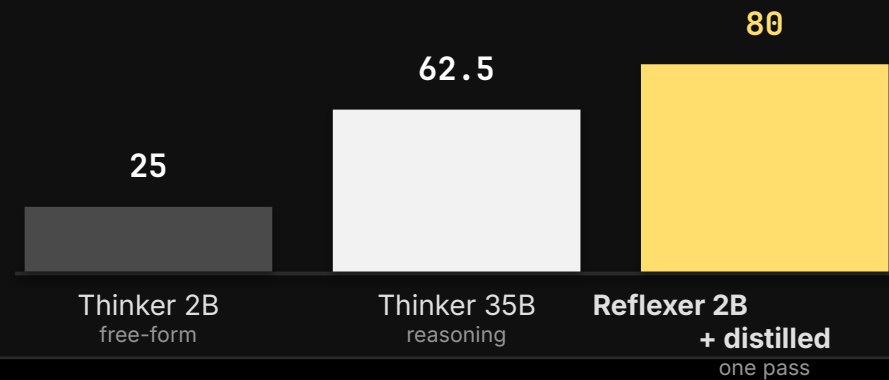
SET-1 • CLEAN API

Efficiency win



SET-2 • REALISTIC ANDROID API

Accuracy win



THE 35B'S FAILURES ARE CONVENTIONS, NOT DISCOVERY

"largest font" **correct: font_scale "1.30"** 35B produced: "1.5"

"DND total silence" **correct: zen_mode 2** 35B produced: 1

It finds the right key by exploring (discovery) — then guesses the value and is wrong the same way every time.

Convention is not discoverable from the API. **Scale rescues discovery; only a distilled pattern carries the convention.**

Result 3 — Teacher quality, economics, and the honest bottleneck

TEACHER QUALITY

25% → 76–80%

- ▶ cold-start distillation: the teacher sees only docs + schema, never a task
- ▶ cloud teacher (Sonnet 4.6): reflexer passes 80% · local 35B teacher: 60%
- ▶ the student faithfully executes whatever the teacher encoded — teacher choice propagates directly

ECONOMICS

break-even $\approx$ 10 calls

- ▶ mint once: ~5–15k teacher tokens · run forever: ~0.3k tokens, <1 s, on-device
- ▶ head intent invoked thousands of times → 100–1000× net win
- ▶ big model spent where it pays: offline teacher + tail fallback

HONEST BOTTLENECK

routing, not binding

- ▶ binding is effectively solved on the deterministic slice (96–100%)
- ▶ routing reaches 66–85%: retrieval is fine (top-3), selection picks the wrong sibling
- ▶ stated plainly: the open problem for a shippable system is the router

Future work — five methodologies, ordered by leverage

1

Session-based device grounding

give the teacher read-only access to the real device, so guessed conventions become observed facts; ground on a development split, never on evaluation gold

2

Deterministic value transforms

move arithmetic and enum lookups (percent → 0–255, label → code) out of the model into a small deterministic function inside the pattern

3

The router: sibling discrimination

retrieval already places the right candidate in the top-3; invest in selection — better selection prompting or a learned selector over behavioural signals

4

Iterative, grounded distillation

run → observe failures → re-distill over 2–3 rounds, lifting a weak local teacher toward cloud quality; the runtime reflex stays a frozen single pass, with a minimality check per round

5

Authoring & personalization at scale

mine a user's repeated trajectories into candidate patterns; privacy-preserving pattern sharing; measure a parametric alternative (tiny fine-tune) against the symbolic approach

Development substrate: all extensions built inside the Pi coding-agent framework — executor and agent loop share one runtime, one tool surface, one instrumentation path.

Thank you.

Questions?



David Zoidze

A.L.F.R.E.D.: Adaptive Local-First Routing and Execution Distillation for Small Language Models

International Black Sea University · Tbilisi · July 2026